US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12399866
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	Huo; Jianjian et al.

Heuristic interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device

Abstract

An interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device comprises: executing a software component at an operating system level in the computer device that monitors update statistics of all data item modifications into the nonvolatile memory device, including one or more of update frequencies for each data item, accumulated update and delete frequencies specific to each file type, and an origin of the data item; storing the update statistics of each of the data items and each of the data item types in a database; and intercepting all operations, including create, write, and update, of performed by applications to all the data items, and automatically assigning a data property identifier to each of the data items based on current update statistics in the database, such that the data items and assigned data property identifiers are transmitted over a memory channel.

Inventors: **Huo; Jianjian (San Jose, CA), Choi; Changho (San Jose, CA), Tseng; Derrick (Union City, CA), Krishnamoorthy; Praveen (Sunnyvale, CA), Huen; Hingkwan (Daly City, CA)**

Applicant: **SAMSUNG ELECTRONICS CO., LTD.** (Suwon-si, KR)

Family ID: **1000008782069**

Assignee: **Samsung Electronics Co., Ltd. (Suwon-si, KR)**

Appl. No.: **18/666711**

Filed: **May 16, 2024**

Prior Publication Data

Document Identifier	Publication Date
US 20240303222 A1	Sep. 12, 2024

Related U.S. Application Data

continuation parent-doc US 17671481 20220214 US 11989160 child-doc US 18666711
continuation parent-doc US 16676356 20191106 US 11249951 20220215 child-doc US 17671481
continuation parent-doc US 15090799 20160405 US 10509770 20191217 child-doc US 16676356
us-provisional-application US 62245100 20151022
us-provisional-application US 62192045 20150713

Publication Classification

Int. Cl.: **G06F16/17** (20190101); **G06F3/06** (20060101); **G06F12/02** (20060101); **G06F16/23** (20190101)

U.S. Cl.:

CPC **G06F16/1727** (20190101); **G06F3/061** (20130101); **G06F3/0619** (20130101); **G06F3/0643** (20130101); **G06F3/0652** (20130101); **G06F3/0679** (20130101); **G06F3/0688** (20130101); **G06F12/0246** (20130101); **G06F16/2365** (20190101);

Field of Classification Search

CPC: G06F (17/30138); G06F (3/0619); G06F (3/0652); G06F (17/30371); G06F (3/0688); G06F (12/0246)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4641197	12/1986	Miyagi	N/A	N/A
4827411	12/1988	Arrowood et al.	N/A	N/A
6282663	12/2000	Khazam	N/A	N/A
6438555	12/2001	Orton	N/A	N/A

6484235	12/2001	Horst et al.	N/A	N/A
6920331	12/2004	Sim et al.	N/A	N/A
7660264	12/2009	Eiriksson et al.	N/A	N/A
7870128	12/2010	Jensen et al.	N/A	N/A
7970806	12/2010	Park et al.	N/A	N/A
8112813	12/2011	Goodwin et al.	N/A	N/A
8312217	12/2011	Chang et al.	N/A	N/A
8495035	12/2012	Aharonov	N/A	N/A
8566513	12/2012	Yamashita	N/A	N/A
8615703	12/2012	Eisenhuth et al.	N/A	N/A
8738882	12/2013	Post et al.	N/A	N/A
8838877	12/2013	Wakrat et al.	N/A	N/A
8874835	12/2013	Davis et al.	N/A	N/A
8996450	12/2014	Rubio	N/A	N/A
9021185	12/2014	Ban	N/A	N/A
9042181	12/2014	Flynn et al.	N/A	N/A
9158770	12/2014	Beadles	N/A	N/A
9213633	12/2014	Canepa et al.	N/A	N/A
9280466	12/2015	Kunimatsu et al.	N/A	N/A
9286204	12/2015	Kikkawa et al.	N/A	N/A
9330305	12/2015	Zhao et al.	N/A	N/A
9413587	12/2015	Smith et al.	N/A	N/A
9575981	12/2016	Dorman et al.	N/A	N/A
9892041	12/2017	Banerjee et al.	N/A	N/A
2001/0016068	12/2000	Shibata	N/A	N/A
2002/0011431	12/2001	Graef et al.	N/A	N/A
2002/0032027	12/2001	Kirani et al.	N/A	N/A
2002/0059317	12/2001	Black et al.	N/A	N/A
2002/0081040	12/2001	Uchida	N/A	N/A
2002/0103860	12/2001	Terada et al.	N/A	N/A
2003/0055747	12/2002	Carr et al.	N/A	N/A
2003/0120952	12/2002	Tarbotton et al.	N/A	N/A
2005/0078944	12/2004	Risan et al.	N/A	N/A
2005/0165777	12/2004	Hurst-Hiller et al.	N/A	N/A
2006/0070030	12/2005	Laborczfalvi	717/120	G06F 9/52
2006/0168517	12/2005	Itoh et al.	N/A	N/A
2006/0242076	12/2005	Fukae et al.	N/A	N/A
2007/0050777	12/2006	Hutchinson et al.	N/A	N/A
2007/0156998	12/2006	Gorobets	N/A	N/A
2007/0225962	12/2006	Brunet et al.	N/A	N/A
2008/0187345	12/2007	Sorihashi	N/A	N/A
2008/0250190	12/2007	Johnson	N/A	N/A
2008/0263579	12/2007	Mears et al.	N/A	N/A
2009/0323022	12/2008	Uchida	N/A	N/A
2010/0017487	12/2009	Patinkin	N/A	N/A
2010/0030822	12/2009	Dawson et al.	N/A	N/A
2010/0146538	12/2009	Cheong et al.	N/A	N/A
2010/0288828	12/2009	Pradhan et al.	N/A	N/A
2011/0066790	12/2010	Mogul et al.	N/A	N/A
2011/0106780	12/2010	Heras et al.	N/A	N/A
2011/0115924	12/2010	Yu et al.	N/A	N/A
2011/0145499	12/2010	Ananthanarayanan et al.	N/A	N/A
2011/0167221	12/2010	Pangal et al.	N/A	N/A
2011/0221864	12/2010	Filippini et al.	N/A	N/A
2011/0246706	12/2010	Gomyo et al.	N/A	N/A
2011/0276539	12/2010	Thiam	N/A	N/A
2012/0007952	12/2011	Otsuka	N/A	N/A
2012/0042134	12/2011	Risan	N/A	N/A
2012/0054447	12/2011	Swart	711/136	G06F 12/0888
2012/0060013	12/2011	Mukherjee	N/A	N/A
2012/0072798	12/2011	Unesaki et al.	N/A	N/A
2012/0096217	12/2011	Son	711/103	G06F 12/0246
2012/0131304	12/2011	Franceschini et al.	N/A	N/A
2012/0150917	12/2011	Sundaram et al.	N/A	N/A
2012/0158827	12/2011	Mathews	N/A	N/A
2012/0191900	12/2011	Kunimatsu et al.	N/A	N/A
2012/0239869	12/2011	Chiueh et al.	N/A	N/A
2012/0254524	12/2011	Fujimoto et al.	N/A	N/A
2012/0278532	12/2011	Bolanowski	N/A	N/A
2012/0317337	12/2011	Johar et al.	N/A	N/A
2012/0323977	12/2011	Fortier et al.	N/A	N/A
2013/0019109	12/2012	Kang et al.	N/A	N/A
2013/0019310	12/2012	Ben-Itzhak et al.	N/A	N/A
2013/0024483	12/2012	Mohr et al.	N/A	N/A
2013/0024559	12/2012	Susanta et al.	N/A	N/A

2013/0024599	12/2012	Huang et al.	N/A	N/A
2013/0050743	12/2012	Steely et al.	N/A	N/A
2013/0097207	12/2012	Sassa	N/A	N/A
2013/0111336	12/2012	Dorman et al.	N/A	N/A
2013/0111547	12/2012	Kraemer	N/A	N/A
2013/0159626	12/2012	Katz et al.	N/A	N/A
2013/0183951	12/2012	Chien	N/A	N/A
2013/0279395	12/2012	Aramoto et al.	N/A	N/A
2013/0290601	12/2012	Sablok et al.	N/A	N/A
2013/0304944	12/2012	Young et al.	N/A	N/A
2014/0049628	12/2013	Motomura et al.	N/A	N/A
2014/0074899	12/2013	Halevy et al.	N/A	N/A
2014/0181499	12/2013	Glod	N/A	N/A
2014/0195921	12/2013	Grosz et al.	N/A	N/A
2014/0208007	12/2013	Cohen et al.	N/A	N/A
2014/0215129	12/2013	Kuzmin et al.	N/A	N/A
2014/0281158	12/2013	Ravimohan et al.	N/A	N/A
2014/0281172	12/2013	Seo et al.	N/A	N/A
2014/0289492	12/2013	Ranjith Reddy	711/170	G06F 3/0613
2014/0333790	12/2013	Wakazono	N/A	N/A
2015/0026257	12/2014	Balakrishnan et al.	N/A	N/A
2015/0058790	12/2014	Kim et al.	N/A	N/A
2015/0074337	12/2014	Jo et al.	N/A	N/A
2015/0113652	12/2014	Ben-Itzhak et al.	N/A	N/A
2015/0146259	12/2014	Enomoto	N/A	N/A
2015/0169449	12/2014	Barrell et al.	N/A	N/A
2015/0186648	12/2014	Lakhotia	N/A	N/A
2015/0188960	12/2014	Alhaidar et al.	N/A	N/A
2015/0278699	12/2014	Danielsson	N/A	N/A
2016/0048445	12/2015	Gschwind et al.	N/A	N/A
2016/0070819	12/2015	Söderberg	N/A	N/A
2016/0078245	12/2015	Amarendran et al.	N/A	N/A
2016/0094603	12/2015	Liao et al.	N/A	N/A
2016/0139838	12/2015	D'Sa et al.	N/A	N/A
2016/0170873	12/2015	Uchigaito et al.	N/A	N/A
2016/0179386	12/2015	Zhang	N/A	N/A
2016/0196076	12/2015	Oh	N/A	N/A
2016/0197950	12/2015	Tsai et al.	N/A	N/A
2016/0203197	12/2015	Rastogi et al.	N/A	N/A
2016/0219024	12/2015	Verzun et al.	N/A	N/A
2016/0239615	12/2015	Dorn	N/A	N/A
2016/0266792	12/2015	Amaki et al.	N/A	N/A
2016/0283125	12/2015	Hashimoto et al.	N/A	N/A
2016/0313943	12/2015	Hashimoto et al.	N/A	N/A
2017/0017663	12/2016	Huo et al.	N/A	N/A
2017/0039372	12/2016	Koval et al.	N/A	N/A
2017/0109096	12/2016	Jean et al.	N/A	N/A
2017/0123666	12/2016	Sinclair et al.	N/A	N/A
2017/0300426	12/2016	Chai et al.	N/A	N/A
2017/0308772	12/2016	Li et al.	N/A	N/A
2017/0339230	12/2016	Yeom et al.	N/A	N/A
2018/0012032	12/2017	Radich et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	**Application Date**	**Country**	**CPC**
103324703	12/2012	CN	N/A
103620549	12/2013	CN	N/A
103842962	12/2013	CN	N/A
103942010	12/2013	CN	N/A
104111898	12/2013	CN	N/A
104391569	12/2014	CN	N/A
104423800	12/2014	CN	N/A
104572491	12/2014	CN	N/A
2-302855	12/1989	JP	N/A
11-327978	12/1998	JP	N/A
2006-235960	12/2005	JP	N/A
2007-102998	12/2006	JP	N/A
2007-172447	12/2006	JP	N/A
2012-104974	12/2011	JP	N/A
2012-170751	12/2011	JP	N/A
2012-020544	12/2012	JP	N/A
2014-167790	12/2013	JP	N/A
2014-522537	12/2013	JP	N/A
2015-8358	12/2014	JP	N/A
2015-5723812	12/2014	JP	N/A

10-2014-0033099	12/2013	KR	N/A
10-2014-0094468	12/2013	KR	N/A
10-2014-0112303	12/2013	KR	N/A
WO 2012/104974	12/2011	WO	N/A
WO 2012/170751	12/2011	WO	N/A
WO 2013/012901	12/2012	WO	N/A
WO 2015/005634	12/2014	WO	N/A
WO 2015/008358	12/2014	WO	N/A
WO 2015/020811	12/2014	WO	N/A

OTHER PUBLICATIONS

Sun, Chao, et al., “SEA-SSD: A Storage Engine Assisted SSD With Application-Coupled Simulation Platform,” IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 62, Issue 1, 2015, 2 pages. cited by applicant
Extended European Search Report issued Aug. 9, 2016 for EP16172142. cited by applicant
Kang et al., The Multi-streamed Solid-State Drive,' HotStorage; 14—6.SUP.th .USENIX Workshop on Hot Topics in Storage and File Systems, Jun. 17-18, 2014. cited by applicant
Kang et al., “The Multi-Streamed Solid State Drive,” Memory Solutions Lab, Memory Division, Samsung Electronics Co., 2014. cited by applicant
Ryu et al., “FlashStream: a multi-tiered storage architecture for adaptive HTTP streaming,” Proceedings of the 21.SUP.st .ACM international conference on Multimedia, ACM, 2013, https://doi.org/10.1145/2502081.2502122. cited by applicant
Stoica et al., “Improving flash write performance by using update frequency.” The 39.SUP.th .International Conference on Very Large Data Bases, Aug. 26-30, 2013, RNa del Garda, Trento, Italy, Proceedings of the VLDB Endowment, vol. 6 No. 9, 733-744. cited by applicant
U.S. Appl. No. 15/090,799, filed Apr. 5, 2016. cited by applicant
Wu et al., “A File System FTL Design for Flash Memory Storage Systems,” EDAA, 978-3-9810801-5-5, Sep. 2009. cited by applicant

---

Primary Examiner: Matin; Tasnima

Attorney, Agent or Firm: Lewis Roca Rothgerber Christie LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) This application is a Continuation application of U.S. patent application Ser. No. 17/671,481, filed Feb. 14, 2022, now U.S. Pat. No. 11,989,160, which is a Continuation application of U.S. patent application Ser. No. 16/676,356, filed Nov. 6, 2019, now U.S. Pat. No. 11,249,951, which claims priority to and the benefit of U.S. patent application Ser. No. 15/090,799 entitled HEURISTIC INTERFACE FOR ENABLING A COMPUTER DEVICE TO UTILIZE DATA PROPERTY-BASED DATA PLACEMENT INSIDE A NONVOLATILE MEMORY DEVICE and filed Apr. 5, 2016, now U.S. Pat. No. 10,509,770, which claims priority to and the benefit of U.S. Provisional Patent Application No. 62/192,045 entitled DATA PROPERTY BASED DATA PLACEMENT IN STORAGE DEVICE and filed Jul. 13, 2015, and U.S. Provisional Patent Application No. 62/245,100 entitled AUTONOMOUS MECHANISM AND ALGORITHM FOR COMPUTER SYSTEM TO UTILIZE MULTI-STREAM SOLID-STATE DRIVE and filed Oct. 22, 2015, the contents all of which are incorporated by reference in their entirety herein.

BACKGROUND
(1) Flash memory based solid-state drives (SSD) have been used widely in both consumer computers and enterprise servers. There are two main types of flash memory, which are named after the NAND and NOR logic gates. NAND type flash memory may be written and read in blocks, each of which comprises a number of pages. Since the NAND flash storage cells in SSDs have very unique properties, SSD's normal usages are very inefficient. For example, although it can be randomly read or programmed a byte or a word at a time, NAND flash memory can only be erased a block at a time. To rewrite a single NAND Flash page, the whole erase block (which contains a lot of flash pages) has to be erased first.
(2) Since NAND flash based storage devices (e.g., SSDs) do not allow in-place updating, a garbage collection operation is performed when the available free block count reaches a certain threshold in order to prepare space for subsequent writes. The garbage collection includes reading valid data from one erase block and writing the valid data to another block, while invalid data is not transferred to a new block. It takes a relatively significant amount of time to erase a NAND erase block, and each erase block has a limited number of erase cycles (from about 3K times to 10K times). Thus, garbage collection overhead is one of the biggest speed limiters in the technology class, incurring higher data I/O latency and lower I/O performance.
(3) Therefore, operating systems (OS) and applications, which don't treat hot/cold data differently, and store them together, will see performance degradation over time (compared to OS's and applications that do treat hot and cold data differently), as well as a shorter SSD lifetime as more erase cycles are needed, causing the NAND cells to wear out faster.
(4) SSD vendors and storage technical committees have come up with a new SSD and standard, called “multi-stream SSD,” to overcome this issue by providing OSs and applications with interfaces that separately store data with different lifespans called “streams.” Streams are host hints that indicate when data writes are associated with one another or have a similar lifetime. That is, a group of individual data writes are a collective stream and each stream is given a stream ID by the OS or an application. For example, “hot” data can be assigned a unique stream ID and the data for that stream ID would be written to the same erase block in the SSD. Because the data within an erase block has a similar lifetime or is associated with one another, there is a greater chance that an entire erase block is freed when data is deleted by a host system, thereby significantly reducing garbage collection overhead because an entire target block would either be valid (and hence no need to erase), or invalid (we can erase, but no need to write). Accordingly, device endurance, and performance should increase.
(5) However, to utilize this new interface, many changes within the applications (including source code) and the OS are required. As a typical computer can have tens or hundreds of software applications installed and running, it's very difficult for all applications, especially for legacy and closed-source applications, to adapt to those changes, in order to use SSDs more efficiently. In addition, multi-stream SSD has limited applicability in that multi-stream SSD is only compatible for use by operating systems and applications.
(6) What is needed is an improved data property based data placement in a storage device, and more particularly, to an autonomous process that enables computer devices to utilize data property based data placement (e.g., multi-stream) solid-state drives.
BRIEF SUMMARY
(7) The example embodiments provide methods and systems for providing an interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device. Aspects of the example embodiments include: executing a software component at an operating system level in the computer device that monitors update statistics of data item modifications into the nonvolatile memory device, including one or more of update frequencies for at least a portion of the data items, accumulated update and delete frequencies specific to each file type, and an origin

of the data item; storing the update statistics for the data items and data item types in a database; and intercepting operations, including create, write, and update, of performed by applications to the data items, and automatically assigning a data property identifier to the data items based on current update statistics in the database, such that the data items and assigned data property identifiers are transmitted over a memory channel to the nonvolatile memory device.

(8) The example embodiments further provide a computer device, comprising: a memory; an operating system; and a processor coupled to the memory, the processor executing a software component provided within the operating system, the software component configured to: monitor update statistics of data item modifications into a nonvolatile memory device, including one or more of update frequencies for at least a portion of the data items, accumulated update and delete frequencies specific to each file type, and an origin of the data item; store the update statistics or the data items and data item types in a database; and intercept all operations, including create, write, and update, of performed by applications to the data items, and automatically assign a data property identifier to each of the data items based on current update statistics in the database, such that the data items and assigned data property identifiers are transmitted over a memory channel to the nonvolatile memory device for storage, thereby enabling the computer device to utilize data property-based data placement inside a nonvolatile memory device.

(9) The example embodiments also provide an executable software product stored on a non-transitory computer-readable storage medium containing program instructions for providing an interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device, the program instructions for: executing a software component at an operating system level in the computer device that monitors update statistics of data item modifications into the nonvolatile memory device, including one or more of update frequencies for at least a portion of the data items, accumulated update and delete frequencies specific to each file type, and an origin of the data item; storing, by the software component, the update statistics for the data items and the data item types in a database; and intercepting all operations, including create, write, and update, of performed by applications to the data items, and automatically assigning a data property identifier to each of the data items based on current update statistics in the database, such that the data items and assigned data property identifiers are transmitted over a memory channel to the nonvolatile memory device for storage.

Description

BRIEF DESCRIPTION OF SEVERAL VIEWS OF THE DRAWINGS

(1) These and/or other features and utilities of the present general inventive concept will become apparent and more readily appreciated from the following description of the embodiments, taken in conjunction with the accompanying drawings of which:

(2) FIG. **1** is a block diagram illustrating an example embodiment of a system for data property-based data placement in a non-volatile memory device provided by a heuristic interface;

(3) FIG. **2** is a flow diagram illustrating the process performed on the host system for providing a heuristic interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device according to one example embodiment;

(4) FIG. **3** is a diagram illustrating the heuristic interface of the example embodiments implemented in various types of computing devices, thereby making the computer devices compatible with an SSD having data property-based data placement;

(5) FIG. **4** is a flow diagram illustrating a process for data property-based data placement performed by the storage controller of the nonvolatile memory device according to an example embodiment;

(6) FIG. **5** is a block diagram illustrating an example of the operation of heuristic interface with respect to storage operations performed by a database application that stores data having different lifetimes and different properties.

DETAILED DESCRIPTION

(7) Reference will now be made in detail to the embodiments of the present general inventive concept, examples of which are illustrated in the accompanying drawings, wherein like reference numerals refer to the like elements throughout. The embodiments are described below in order to explain the present general inventive concept while referring to the figures.

(8) Advantages and features of the present invention and methods of accomplishing the same may be understood more readily by reference to the following detailed description of embodiments and the accompanying drawings. The present general inventive concept may, however, be embodied in many different forms and should not be construed as being limited to the embodiments set forth herein. Rather, these embodiments are provided so that this disclosure will be thorough and complete and will fully convey the concept of the general inventive concept to those skilled in the art, and the present general inventive concept will only be defined by the appended claims. In the drawings, the thickness of layers and regions are exaggerated for clarity.

(9) The use of the terms “a” and “an” and “the” and similar referents in the context of describing the invention (especially in the context of the following claims) are to be construed to cover both the singular and the plural, unless otherwise indicated herein or clearly contradicted by context. The terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (i.e., meaning “including, but not limited to,”) unless otherwise noted.

(10) The term “algorithm” or “module”, as used herein, means, but is not limited to, a software or hardware component, such as a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC), which performs certain tasks. An algorithm or module may advantageously be configured to reside in the addressable storage medium and configured to execute on one or more processors. Thus, an algorithm or module may include, by way of example, components, such as software components, object-oriented software components, class components and task components, processes, functions, attributes, procedures, subroutines, segments of program code, drivers, firmware, microcode, circuitry, data, databases, data structures, tables, arrays, and variables. The functionality provided for the components and components or modules may be combined into fewer components or modules or further separated into additional components and components or modules.

(11) Unless defined otherwise, all technical and scientific terms used herein have the same meaning as commonly understood by one of ordinary skill in the art to which this invention belongs. It is noted that the use of any and all examples, or exemplary terms provided herein is intended merely to better illuminate the invention and is not a limitation on the scope of the invention unless otherwise specified. Further, unless defined otherwise, all terms defined in generally used dictionaries may not be overly interpreted. In one aspect, the example embodiments provide a heuristic and

(12) autonomous interface for enabling computer systems to utilize a data property-based data placement method (e.g., multi-streaming) in storage devices, such as SSDs, which does not require changes to applications.

(13) FIG. **1** is a block diagram illustrating an example embodiment of a system for data property-based data placement in a nonvolatile memory device provided by a heuristic interface. The example embodiments described herein may be applicable to any nonvolatile memory device requiring garbage collection, and will be explained with respect to an embodiment where the nonvolatile memory device comprises a solid-state drive (SSD).

(14) The system includes a host system **10** coupled to an SSD **12** over a channel **14**. As is well-known, an SSD has no moving parts to store data and does not require constant power to retain that data. Components of the host system **10** that are relevant to this disclosure include a processor **16**, which executes computer instructions from a memory **18** including, an operating system (OS) **20** and a file system **21**. The host system **10** may include other components (not shown), such as a memory controller for interfacing with the channel **14**. The host system **10** and the SSD **12** communicate commands and data items **26** over the channel **14**. In one embodiment, the host system may be a typical computer or server running any type of OS. Example types of OSs include single- and multi-user, distributed, templated, embedded, real-time, and library. In another embodiment, the system may be a standalone component, such as a device controller, in which case the OS may comprise a lightweight OS (or parts thereof) or even firmware.

(15) The SSD **12** includes a storage controller **22** and a nonvolatile memory (NVM) array **24** to store data from the host system **10**. The storage controller manages **22** the data stored in the NVM array **24** and communicates with the host system over the channel **14** via communication protocols. The NVM array **24** may comprise any type of nonvolatile random-access memory (NVRAM) including flash memory, ferroelectric RAM (F-RAM), magnetoresistive RAM (MRAM), phase-change memory (PCM), millipede memory, and the like. Both the SSD **12** and channel **14** may support multi-channel memory architectures, such as dual channel architecture; and may also support single, double or quad rate data transfers.
(16) According to the example embodiments, in order to reduce garbage collection overhead in the SSD **12**, the example embodiments provide an improved data property-based data placement in the SSD. This is accomplished by providing a heuristic interface **26** that enables both applications and hardware components to separately store data items in the SSD **12** that have different lifespans. In addition, in some embodiments, use of a heuristic interface **26** requires no changes to user applications running on the host system **10**.
(17) In one embodiment, the heuristic interface **26** comprises at least one software component installed at the operating system level that continuously monitors and stores usage and update statistics of all data items **28**, such as files. After an initial warm-up or training period, any create/write/update operation performed on the data items **28** by the host system **10** are assigned a dynamic data property identifier **30** according to current usage and update statistics. In one embodiment, the actual assignment of the data property identifiers **30** may be performed by software hooks in the file system **21** of the OS **20**.
(18) FIG. **2** is a flow diagram illustrating the process performed on the host system for providing a heuristic interface for enabling a computer device to utilize data property-based data placement inside a nonvolatile memory device according to one example embodiment. The process may include executing a software component (e.g., the heuristic interface) at an operating system level in the computer device that monitors update statistics of data item (e.g., file) modifications into the nonvolatile memory device, including update frequencies for at least a portion of the data items, accumulated update and delete frequencies specific to each file type, and an origin of the data file (block **200**).
(19) The software component stores the update statistics for the data items and data item types in a database (block **202**). In one embodiment, the update statistics may be stored both in the host system memory **18** and in the SSD **12**.
(20) The software component intercepts most, if not all, operations, including create, write, and update, performed by applications to the data items, and automatically assigns a data property identifier to each of the data items based on current update statistics in the database, such that the data items and assigned property identifiers are transmitted over the memory channel to the nonvolatile memory device for storage (block **204**). In one embodiment, the data property identifier acts as a tag, and need not actually transmit any information on what data property the identifier represents.
(21) According to one embodiment, the heuristic interface **26** uses the current update statistics to associate or assign the data property identifiers **30** to each of the data items **28** based on one or more data properties indicating data similarity, such as a data lifetime, a data type, data size, and a physical data source. In a further embodiment, logical block address (LBA) ranges could also be used as a data property indicating data similarity. For example, a pattern of LBA accesses could be an indicator of similarity and call for a grouping of the data. In this manner, data items **28** having the same or similar data properties are assigned the same data property identifier value.
(22) Because the heuristic interface **26** is provided at the operating system level, no changes are required to existing applications in order to make those applications compatible with the data property-based data placement process of the example embodiment. Accordingly, the heuristic interface **26** may be implemented in any type of computing device having a processor and operating system to expand use of conventional multi-streaming beyond applications and operating systems.
(23) FIG. **3** is a diagram illustrating the heuristic interface of the example embodiments implemented in various types of computing devices **300**A-**300**D, thereby making the computer devices compatible with an SSD **12** having data property-based data placement (e.g., multi-streaming). Computing devices **300**A and **300**B may represent a host device such as a PC or server or storage subsystem in which respective heuristic interfaces **26**A and **26**B are provided within operating systems **20**A and **20**B. The heuristic interface **26**A intercepts data item operations performed by an application **304**A, and automatically assigns data property identifiers **30**A to each of the data items from application **304**A based on current update statistics. Similarly, the heuristic interface **26**B intercepts data item operations performed by a block layer **23**B of the OS **20**B, and automatically assigns data property identifiers **30**B to each of the data items operated on by the block layer **23**B based on current update statistics.
(24) Computing devices **300**C and **300**D may represent hardware devices, such as a switch, a router, a RAID system, or a host bus adapter (HBA) system, a sensor system or stand-alone device (such as a scanner or camera) in which respective heuristic interfaces **26**C and **26**D are provided within hardware device controllers **306**C and **306**D. The heuristic interface **26**C intercepts data item operations performed by the device controller **306**C, and automatically assigns data property identifiers **30**C to each of the data items from the device controller **306**C based on current update statistics. Similarly, the heuristic interface **26**D intercepts data item operations performed by the device controller **306**D, and automatically assigns data property identifiers **30**D to each of the data items from the device controller **306**D based on current update statistics.
(25) FIG. **4** is a flow diagram illustrating a process for data property-based data placement performed by the storage controller of the nonvolatile memory device according to an example embodiment. Referring to both FIGS. **3** and **4**, the process may include receiving over a channel from at least one of an operating system or an executing application (e.g., OS **20**B or apps **304**A), a first series of data items to be stored, wherein each of the data items includes a first data property identifier that is associated with the data items based on the one or more data properties indicating data similarity, which may include the data lifetime, the data type, data size, and the physical data source (block **400**).
(26) In one embodiment, the data type may include properties outside of the application of origin. In a further embodiment, the one or more data properties could also include logical block address (LBA) ranges. It should be noted the data property listed here are not the only types of data properties that can be considered. For example, some other data property, which is currently not known, could be used to assign determine data similarity in the future. In one embodiment, the data items received from the operating system or the application may include data property identifiers that are associated with the data items by another process besides the heuristic interface **26**.
(27) The storage controller receives over the channel from a hardware device controller (e.g., controllers **306**C and **306**D) another series of data items to be stored, wherein each of the data items includes a second data property identifier that is associated with the data items based on one or more of the data properties indicating data similarity, including a data lifetime, a data type, and a physical data source (block **402**).
(28) The storage controller reads the data property identifiers and identifies which blocks of the memory device to store the corresponding data items, such that the data items having the same data property identifiers are stored in a same block (block **404**), and stores the data items into the identified blocks (block **406**).
(29) For example, with respect to FIG. **3**, the storage controller **22** of the SSD **12** receives the data items from the computing devices **300**A and **300**B that have been assigned data property ID **30**A and **30**B, respectively. The storage controller **22** then stores data items associated with data property ID **30**A within the same erase block(s) **308**A, and stores data items associated with data property ID **30**B within the same erase block(s) **308**B. Similarly, the storage controller **22** of the SSD **12** receives the data items from the device controllers **306**C and **306**D of computing devices **300**C and **300**D that have been assigned data property ID **30**C and **30**D, respectively. The storage controller **22** then stores data items associated with data property ID **30**C within the same erase block(s) **308**C, and stores data items associated with data property ID **30**D within the same erase block(s) **308**D.
(30) According to the heuristic interface **26** of the example embodiments, a data property ID **30** may be assigned to the data items **28** that are output from any type input device for storage. For example, assume the heuristic interface **28** is implemented within a digital security camera that takes an image periodically, e.g., once a second. The heuristic interface **28** may assign a data property ID **30** to each image file based on data properties indicating data similarity, such as the capture rate of the images, the image file size, and the origin of the images (e.g., device ID and/or GPS location). Note that such data properties need not be associated with a particular application or file system as is the metadata used by conventional multi-streaming.

(31) FIG. **5** is a block diagram illustrating the heuristic interface, which is one example of an identifier assignment process, in further detail. Similar to FIGS. **1** and **3**, the heuristic interface **26** is shown provided at the operating system **20** level of the host system **10**. The heuristic interface **26** monitors the operations of the applications **300**, the file system **21**, and block layer **23** and device drivers, and assigns data property identifiers **30** to data items based on data properties in the update statistics indicating data similarity. The SSD **12** reads the data property identifiers **30** and identifies which blocks of the SSD **12** are used to store the corresponding data items so that the data items having the same data property identifiers **30** are stored in a same block.
(32) According to some example embodiments, the heuristic interface **26** comprises two software components: a stats demon **600** installed within the operating system **20**, and map hooks **602** implemented as system call hooks at the file system level. The stats demon **600** may continuously run in the background to manage and maintain at least one heuristic/statistical database **604** of data item modifications to the SSD **12** (shown in the expanded dotted line box). The map hooks **602** may intercept all file update operations and automatically assign a data property ID **30** to each of the file update operations **610** according to current statistics in the database **604**.
(33) In one embodiment, the statistics stored in the heuristic database **604** may comprise two types, filename-based statistics and file type-based statistics. The filename-based statistics may record update frequencies (includes rewrite, update, delete, truncate operations) for each data file. The file type-based statistics may record accumulated update and delete frequencies for specific file types. In one embodiment, both sets of statistics are updated by each file update, and may be stored in both memory **18** and on the SSD **12**.
(34) In one embodiment, the heuristic database **604** may store the two types of statistics using respective tables, referred to herein as a filename-based statistics table **606** and a file type-based statistics table **608**. In one embodiment, the tables may be implemented as hash tables. Each entry in the filename-based statistics table **606** may include a key, which may be a file name, and a value, which may comprise a total number of updates (includes rewrite, update, truncate) for the file during its whole lifetime. Responsive to the creation of a new file, the stats daemon **600** may create a new entry to the filename-based statistics table **606**. Responsive to the file being deleted, stats daemon **600** may delete the corresponding entry in filename-based statistics table **606**.
(35) Each entry in the file type-based table **608** may include a key, which may be the file type (mp3, mpg, xls and etc.), and a value, which may comprise all total updates (includes rewrite, update, truncate and delete) made to files of this type divided by the number of files of this file type. Responsive to the creation of a new type, the stats daemon **600** may create a new entry in the file type-based statistics table **608**, and the stats demon **600** may not delete any entry after the entry is added.
(36) The stats demon **600** also is responsible for loading these two hash tables from SSD **12** to the memory **18** after operating system boot up, and flushes the tables to the SSD periodically for permanent storage. Both hash tables can be stored in the SSD **12** as normal data files.
(37) For a newly installed operating system, the heuristic interface **26** may require a configurable warm-up period to collect sufficient statistics to effectively assign data property identifiers **30**.
(38) The map hooks **602** may be implemented as system call hooks at the file system level to intercept all file update operations (create/write/update/delete) **610** made to all files by the applications **300** and the OS **20**. Most operating systems, such as Windows and Linux, provide file system hooks for system programming purposes.
(39) Responsive to intercepting file update operations **610**, the map hooks **602** may be performed by a heuristic database update operation **612** to send related information to the stats daemon **600**. The map hooks **602** then performs a data property ID calculation via block **614** that reads the heuristic database **604**, and calculates and assigns a data property identifier **30** for the actual file writes according to current statistics via line **616**. File creations or fresh file writes may be assigned data property IDs **30** according to the file's type and current file type statistics; while the file data updates may be assigned data property IDs **30** according to its update frequency and current update frequencies statistics. Finally, the map hooks **602** forwards the actual file writes to the underlying file system **21** via block **618**.
(40) The map hooks **602** may use an algorithm based on two statistics hash tables **606** and **608** to calculate and assign data property IDs, below is a simplified example implementation. The hash table lookup and calculations overhead can be very minimal, thus the SSD read/write throughput won't be affected. More information can be added into these statistics hash tables **606** and **608**, and more complicated stream ID calculation algorithms can be used.
(41) In one embodiment, an example simplified data property ID calculation algorithm is a follows: 1. Intercept file creation: add a new entry in the filename-based statistics table; if it's a new type of file, add a new entry in the filetype-based statistics table too. 2. Intercept file read: do nothing. 3. Intercept file delete: delete the file's entry in the filename-based statistics table. 4. Intercept fresh file write: If past warm-up period, calculate stream ID as below; otherwise, assign lowest stream ID available. Data property ID=floor ((Write_Frequency_This_FileType/Max_Write_Frequency_in_FileType_Hash Table)×Number_of_Available_Streams_in_NVM_Device) Assign stream ID to actual file write and forward it to file system and SSD. 5. Intercept file update: Update the filename-based statistics table by increasing the update times in this file's entry. Update the filetype-based statistics table by increasing the accumulated write frequency in this file type's entry. If past warm-up period, calculate data property ID as below; otherwise, assign lowest data property ID available. Data property ID=floor ((Total_Writes_This_File/Max_Writes_in_FileName_HashTable)×Number_of_Available_Streams_in_NVM_Device)
(42) As an example of the data property ID calculation, assume the following: a user is editing a photo app to edit a photo file named foo.jpg (a JPEG file type); and the SSD of the user's computer is configured to handle up to four data property IDs or stream IDs. When the user saves the photo onto the SSD **12**, the map hooks **602** intercept the file save request, determines the file type and updates the update frequency of that particular file and to the JPEG file type in the heuristic database **604**.
(43) The maps hooks **602** also search the current statistics for the write frequency (e.g., per day) for the JPEG file type as well as the maximum write frequency over all file types. In this example, assume that that the JPEG file type has a write frequency per day of 10 and that a ".meta" file type has a write frequency per day of 100, which is the highest or maximum write frequency over all file types. The map hooks **602** may then calculate the data property ID to be assigned to the file save operation using the equation:
(44)

$$\text{Dataproperty}ID = \text{floor}((\text{Write\_Frequency\_This\_fileType} / \text{Max\_Write\_Frequency\_in\_FileType\_HashTable}) \times \text{Number\_of\_Available\_Streams\_in\_NVM\_De}$$

$$\text{Dataproperty}ID = \text{floor}((10 / 100) \star 4) + 1 = 1$$

(45) The data property ID of 1 is then sent with the file data through the file system **21** and block layer **23** to the SSD **12** for storage and block that stores other data having an assigned data property ID of 0.
(46) In an alternative embodiment, the heuristic interface **26** (i.e., the stats demon **600** and the map hooks **602**) and assignment of data property IDs based on update statistics in the heuristic database **604**, may be implemented within the OS block storage layer **23** or even inside the SSD **12**. For both cases, the stats demon **604** only needs to maintain a lookup table to store update statistics for each storage block, and the map hooks **602** may calculate and assign the data property IDs to each block update according to update frequencies stored in the lookup table. In the case of implementation inside the OS block storage layer **23**, the map hooks **602** may be implemented in the OS block storage layer instead of the file system **21**, the stats demon **604** may be a kernel mode daemon which is part of operating system, and the statistic tables **606** and **608** can be stored in both memory and a specific partition inside SSD **12**. In the case of implementation inside a data property-based SSD **12**, the statistics tables **606** and **608** can be stored in a NAND flash memory spare area to save storage space, and the in-memory part of the statistics tables **606** and **608** can be stored and merged with current SSD FTL mapping tables.
(47) In one embodiment, the heuristic interface **26** is implemented as a software component. In another embodiment, the heuristic interface **26** could be implemented as a combination of hardware and software. Although the stats demon **600** and the map hooks **602** are shown as separate

components, the functionality of each may be combined into a lesser or a greater number of modules/components. For example, in another embodiment, the stats demon **600** and the map hooks **602** may be implemented as one integrated component.
(48) The heuristic interface **26** of the example embodiments may be applied to a broad range of storage markets from client to enterprise, which could be applied to a disk for a single standalone machine (such as desktop, laptop, workstation, server, and the like), storage array, software-define storage (SDS), application-specific storage, virtual machine (VM), virtual desktop infrastructure (VDI), content distribution network (CDN), and the like.
(49) In one embodiment, for example, the NVM array **24** of the SSD **12** may be formed of a plurality of non-volatile memory chips, i.e., a plurality of flash memories. As another example, the NVM array **24** may be formed of different-type nonvolatile memory chips (e.g., PRAM, FRAM, MRAM, etc.) instead of flash memory chips. Alternatively, the array **24** can be formed of volatile memories, i.e., DRAM or SRAM, and may have a hybrid type where two or more types of memories are mixed.
(50) A methods and systems for a heuristic and autonomous interface for enabling computer systems to utilize the data placement method has been disclosed. The present invention has been described in accordance with the embodiments shown, and there could be variations to the embodiments, and any variations would be within the spirit and scope of the present invention. For example, the exemplary embodiment can be implemented using hardware, software, a computer readable medium containing program instructions, or a combination thereof. Software written according to the present invention is to be either stored in some form of computer-readable medium such as a memory, a hard disk, or a CD/DVD-ROM and is to be executed by a processor. Accordingly, many modifications may be made by one of ordinary skill in the art without departing from the spirit and scope of the appended claims.

Claims

1. A method comprising: monitoring, by a processor of a computing device, data items allocated for a nonvolatile memory device of the computing device; assigning, by the processor, a data property identifier to a data item of the data items, the data property identifier indicating data similarity and is based on at least one of a data lifetime, a data type, a data size, or a physical data source; and transferring, by the processor, the data items and the assigned data property identifiers to the nonvolatile memory device for storage.
2. The method of claim 1, wherein a statistic of data items modification comprises an update statistic for the data items and data item types, and wherein the data lifetime or the data type is based on the update statistic from the statistic of data items modification.
3. The method of claim 2, wherein the statistic of data items modification comprises logical block address (LBA) access range statistics for the data item and wherein the data type or the data size is based on the LBA access range statistics of the data item.
4. The method of claim 2, wherein the method further comprising: storing, by the processor, the update statistic for the data items and the data item types in a database; and intercepting create, write, and update operations, performed by applications running in the processor to the data items.
5. The method of claim 2, wherein data properties indicate data similarities and comprises one or more of the data type, the data size, logical block address (LBA) ranges, LBA access patterns, and the physical data source.
6. The method of claim 2, wherein the processor comprises a software component comprising: a daemon installed in an operating system of the computing device; and system call hooks implemented at a file system level, wherein the software component is executed within a device controller of the computing device, and wherein the daemon continuously runs in background to manage and maintain a database of the data items modification to the nonvolatile memory device.
7. The method of claim 6, wherein the data items comprise files and wherein the statistic of data items modification comprises a filename-based statistic and a file type-based statistic, wherein the filename-based statistic records update frequencies, comprising rewrite, update, delete, and truncate operations, for the data items, and wherein the file type-based statistic records accumulated update and delete frequencies specific file types, wherein a portion of the filename-based statistic and the file type-based statistic are stored in both memory of the computing device and on the nonvolatile memory device, and wherein the filename-based statistic and the file type-based statistic are implemented as hash tables.
8. The method of claim 7, wherein an entry in a hash table of the hash tables based on the filename-based statistic comprises a key and a value, wherein the value comprises a total number of updates for the files during a lifetime of the files.
9. The method of claim 8, wherein responsive to creation of a new file, the daemon creates a new entry in the hash table based on the filename-based statistic; and responsive to the files being deleted, the daemon deletes a corresponding entry in the hash table based on the filename-based statistic.
10. The method of claim 7, wherein an entry in a hash table of the hash tables based on the file type-based statistic comprises a key and a value, wherein the value comprises all total updates made to files of a type divided by a number of files of the file type-based statistic.
11. The method of claim 10, wherein responsive to creation of a new type, the daemon creates a new entry in the hash table based on the file type-based statistic; and the daemon does not delete any entry after the entry is added.
12. The method of claim 7, wherein the daemon loads the filename-based statistic and the file type-based statistic to the memory of the computing device after operating system boot up, and flushes the filename-based statistic and the file type-based statistic to the nonvolatile memory device periodically for permanent storage.
13. The method of claim 7, wherein the system call hooks intercept all file update operations and automatically assign a dynamic property identifier to a file update operation of the file update operations according to current statistic in the database.
14. The method of claim 7, wherein the system call hooks are configured to: responsive to intercepting file update operations, perform a database update operation to send related information to the daemon; perform a data property identifier calculation that reads the database and calculates and assigns a data property identifier to actual file writes according to a current statistic, wherein file creations or fresh file writes are assigned data property identifiers according to a file type and current file type statistic; wherein file data updates are assigned data property identifiers according to its update frequency and a current update frequencies statistic; and forward the actual file writes to a file system of the file system level.
15. The method of claim 14, wherein the nonvolatile memory device is configured to handle a maximum number of streams and wherein the data property identifier calculation responsive to the fresh file writes comprises assigning a data property identifier equal to a quantity rounded down to a lowest integer, the quantity being a write frequency for the file type divided by a maximum write frequency from the current file type statistic multiplied by a number of available streams in the nonvolatile memory device, the number of available streams being equal to the maximum number of streams minus a current number of streams.
16. The method of claim 14, wherein the nonvolatile memory device is configured to handle a maximum number of streams and wherein the data property identifier calculation responsive to a file update comprises assigning a data property identifier equal to a quantity rounded down to a lowest integer, the quantity being a total number of writes for the files divided by a maximum number of writes for the files from the current file type statistic multiplied by a number of available streams in the nonvolatile memory device, the number of available streams being equal to the maximum number of streams minus a current number of streams.
17. The method of claim 7, further comprising: responsive to a newly installed operating system, requiring the software component to have a configurable warm-up period to collect sufficient statistics to effectively assign data property identifiers.
18. A computer device, comprising: a memory; and processor coupled to the memory, the processor being configured to: monitor data items allocated for a nonvolatile memory device; assign a data property identifier to a data item of the data items, the data property identifier indicating data similarity and is based on at least one of a data lifetime, a data type, a data size, or a physical data source; and transmit the data items and the assigned data property identifiers to the nonvolatile memory device for storage.

19. The computer device of claim 18, wherein a statistic of data items modification comprises an update statistic for the data items and data item types, wherein the data lifetime or the data type is based on the update statistic from the statistic of data items modification, and wherein the processor is further configured to: store the update statistic for the data items and the data item types in a database; and intercept create, write, and update operations, performed by applications running in the processor to the data items.

20. A method comprising: monitoring, by a processor of a computing device, data items allocated for a nonvolatile memory device of the computing device; assigning, by the processor, a data property identifier to a data item of the data items, the data property identifier indicating data similarity based on at least one of a data lifetime, a data type, a data size, or a physical data source; and transferring, by the processor, the data items and the assigned data property identifiers to the nonvolatile memory device for storage, wherein a statistic of data items modification comprises an update statistic for the data items and data item types, and wherein the data lifetime or the data type is based on the update statistic from the statistic of data items modification.